MICROSOFT FABRIC REAL-TIME PHARMA ANALYTICS PROJECT

Real-Time Prescription Monitoring & Drug Diversion Detection (Lambda Architecture)

Table of Contents

1. Project Objective & Lambda Architecture	5. Cold Path: Historical Lakehouse ML
2. Provisioning Workspace & Resources	6. Data Activator Real-Time Remediation
3. Python Synthetic Rx Stream Simulator	7. Executive Direct Lake Power BI Report
4. Hot Path: KQL Database & Queryset Rules	8. Operational Real-Time KQL Dashboard

1. Project Objective & Lambda Architecture

This hands-on data engineering and AI solution implements a dual-path Lambda streaming architecture in Microsoft Fabric to detect fraudulent prescription activities, doctor-shopping velocity, and opioid drug diversion across healthcare distribution networks in real-time.

- **Bronze Ingestion Layer**: High-throughput ingestion of e-Prescription (eRx) payloads via Fabric Eventstream.
- **Silver Hot Path (Speed Layer)**: Real-Time KQL Database executing sub-second sliding-window pattern detection.
- **Silver Cold Path (Batch Layer)**: Lakehouse Delta tables scored by a Spark ML Random Forest model.
- **Gold Action Layer**: Instantaneous alerting via Data Activator (Reflex) and executive Direct Lake reporting.

2. Provisioning & Ingestion Setup

- Create Workspace: RealTime_Pharma_Diversion_Detection
- Create Lakehouse: Pharma_Compliance_Ops
- Create Eventhouse: RealTime_Pharma_DB
- Create Eventstream: Rx_Dispense_Stream with a Custom Endpoint named PharmacyGatewayConnector.
- Copy the **Event Hub Name** and **Connection String-Primary Key** from SAS Key Authentication settings.

3. Hot Path: Real-Time KQL Queryset

Execute the DDL in **RealTime_Pharma_DB** Queryset:

```
.create table RawPrescriptions ( rx_id: long, patient_id: string, hcp_dea_id: string, drug_name: string, units_dispensed: real, location: string, timestamp: datetime, anomaly_type: string, risk_score: real )
```

Hot Path Rule 1: Doctor Shopping & Rapid Multi-Pharmacy Dispensing (10s Window)

```
RawPrescriptions | where units_dispensed > 0 | summarize rx_count = count(), total_units = sum(units_dispensed) by patient_id, bin(timestamp, 10s) | where rx_count > 3 | project timestamp, patient_id, rx_count, total_units, AlertReason = "Doctor Shopping Velocity Attack";
```

Hot Path Rule 2: Impossible Travel & Geographic Translocation (Under 2 Mins)

```
RawPrescriptions | order by patient_id asc, timestamp desc | extend prev_location = prev(location), prev_time = prev(timestamp), prev_patient = prev(patient_id) | where patient_id == prev_patient and location != prev_location | extend time_diff_sec = datetime_diff('second', prev_time, timestamp) | where time_diff_sec between (1 .. 120) | project timestamp, patient_id, CurrentLocation = location, PreviousLocation = prev_location, time_diff_sec, AlertReason = "Cross-State Geographic Translocation";
```

4. Cold Path: Spark ML Model Deployment

Route Eventstream destination to Lakehouse Pharma_Compliance_Ops as Delta table RawPrescriptions. Train a 20-Tree Random Forest model inside a Spark Notebook:

```
from pyspark.sql.functions import col, when from pyspark.ml.feature import StringIndexer, VectorAssembler from pyspark.ml.classification import RandomForestClassifier df_raw = spark.read.table("Pharma_Compliance_Ops.RawPrescriptions") indexer_loc = StringIndexer(inputCol="location", outputCol="loc_idx").fit(df_raw) indexer_drug = StringIndexer(inputCol="drug_name", outputCol="drug_idx").fit(df_raw) df_indexed = indexer_drug.transform(indexer_loc.transform(df_raw)) assembler = VectorAssembler(inputCols=["units_dispensed", "risk_score", "loc_idx", "drug_idx"], outputCol="features") df_features = assembler.transform(df_indexed) df_data = df_features.withColumn("label", when(col("anomaly_type") != "None", 1.0).otherwise(0.0)) rf = RandomForestClassifier(labelCol="label", featuresCol="features", numTrees=20) model = rf.fit(df_data) predictions = model.transform(df_features) df_alerts = predictions.filter(col("prediction") == 1.0).select( col("rx_id"), col("patient_id"), col("units_dispensed").alias("FlaggedUnits"), col("drug_name"), col("location"), col("timestamp"), col("anomaly_type").alias("DiversionType") ) df_alerts.write.mode("overwrite").format("delta").saveAsTable("Pharma_Compliance_Ops.diversion_alerts")
```

5. Data Activator (Reflex) & Serving Layer

A. Automated Alerting via Data Activator:

- In KQL Queryset, highlight Rule 1 & click **Add alert**.
- Alert Name: Doctor_Shopping_Reflex_Trigger (Run every 5 mins on each event).
- Subject: CRITICAL PHARMA COMPLIANCE: Rapid Opioid Dispensing Detected.
- Context: Dynamically inject patient_id and total_units into email body.

B. Power BI Direct Lake Report:

- Model: Gold_Pharma_Compliance_Semantic_Model (Tables: diversion_alerts, RawPrescriptions).
- **Card 1**: Total Incidents = COUNT(rx_id).
- **Card 2**: Loss Exposure (Flagged Units) = SUM(FlaggedUnits).
- **Donut Chart**: Attack Types Breakdown (Legend: DiversionType, Values: COUNT(rx_id)).
- **Bar Chart**: Geographic Threat Exposure (Y-axis: location, X-axis: FlaggedUnits).

C. Real-Time KQL Operational Dashboard:

- Pin Rule 1 (Live Velocity Attacks) and Rule 2 (Live Geographic Translocations) directly from KQL Queryset to a Real-Time Dashboard.
- Set **Refresh settings** to **Live refresh** for dynamic sub-second operational monitoring.